

ATNLP Rapport

Mariusz Matyja
University of Copenhagen

1 Introduction

Researchers continuously review previous research in light of discoveries. This phenomenon also occurs in the field of machine learning. Since the introduction of Transformer architecture for neural networks, many researchers have attempted to verify how well the new models can handle tasks previously approached using older architectures. This work revolves around the same principle.

This work aims to reproduce the experiments in *Generalization without Systematicity* using the Transformer architecture. The named paper investigates how and to what degree RNN networks can generalize to new, rarely seen words. The paper presents the following, as an example of such generalization: If a person knows the meaning of word "dax", it will not be problematic for the person to use the word in a context (more or less complicated) in which the person has not yet seen the word. For instance, "dax twice", or "dax thrice". The experiments in said paper revolve around an autoregressive supervised sequence-to-sequence text generation/translation task.

To perform the experiments, two different transformer architectures are used. One is a standard architecture, which will not be pretrained on data outside of the scope (denoted as *vanilla models*). The other architecture is BART. The latter will be initialized from a pretrained checkpoint, and only finetuned using PEFT methods.

The main hypothesis is, that the transformer architecture will perform in general better, than RNN networks. The other assumption is, that the BART models will not yield as good results, as the vanilla models, given the specific vocabulary of the SCAN dataset.

2 Methods

2.1 Dataset

The dataset used in this work is the same dataset, as in the original paper i.e. SCAN dataset. The dataset consists of 20 thousands of input/target sequence pairs i.e. the data is fully labeled. The vocabulary of the source and target sequences has no common words. All input sequences are a combination of primitive commands, and modifiers, whereas the output sequences consist only of primitives. For instance, input sequence "turn left twice" translates to "LTURN LTURN", where "turn left", and "LTURN" is a primitive command, and "twice" is a modifier. The dataset consists of different, each used for a particular part of one of the experiments.

2.2 Experiments

Three experiments are reproduced in this work E1, E2, and E3.

- E1: Generalizing to a random subset of commands
The models are trained on various sized of random subsets of the dataset. The used sizes in percent of the whole dataset are: 1,2, 4,8,16,32,64.
- E2: Generalizing to commands demanding longer action sequences
The dataset used for model training consists of approx. 17 000 sequence pairs of length up to 22 output actions. The test set consists of the remaining pairs of longer target sequences up to 48 actions.
- E3 Generalizing composition across primitive commands
In this experiment models train on dataset containing various amounts of a particular primitive.
 - E3.1 ("turn left") The split consists of only few examples of "turn left"
 - E3.2 ("jump") Multiple splits containing various amounts of "jump" primitive.

2.3 Vanilla Models

The vanilla models were implemented using template, as well as the original transformer paper. The

hyperparameters were decided upfront, thus there was no need for a hyper-parameter search. The only exception (later explained in details) was the number of training steps. See Appendix 1, for extensive list of the predefined hyperparameters.

2.3.1 Data Preprocessing

Single vocabulary was created for both input, and output languages. The sequences were split into tokens, where each token represented a single word. The token lists were trimmed, or padded to unified sequence lengths: 75 tokens for input sequences, and 80 for output sequences. Three special tokens were used: "PAD", "SOS", and "EOS": padding, start of sequence, end of sequence. There was no need to introduce a token representing unknown words, since the whole vocabulary was known.

2.3.2 Training and Metrics

The training regime differed from the one described in the original work. There were two main changes: increased number of training steps, and use on teacher forcing only, instead of using it only for half of the training steps. The changes were introduced due to critically low accuracies obtained when training models for experiment E1, which in the original paper was 'the easiest' for all models. The initial assumption was, that the transformer architecture being more complicated than RNN requires more training to converge, and due to the specific nature of the dataset, i.e. primitives and modifiers translate to primitives only, teacher forcing was the only working technique.

Unfortunately, later the assumptions showed to be false, when a simple bug in the implementation of the architecture was found. Due to time constraints the training regime was not changed to match the original paper, but based on previous results a fixed number of 50 epoches for each model was chosen. However it is possible, that the changes had no major influence on the final results.

In order to obtain comparable results the metrics collected were accuracy scores both token-wise, and sequence-wise: same as in the the original paper.

2.3.3 Experimentation Process

As mentioned, accuracy scores obtained for initial models training were critically low, therefore the training regime was changed. In order to obtain an optimal number of training steps k-fold cross-validation was performed. Unfortunately, even with

greatly larger number of steps (between 10 and 20 times more) the results were not satisfactory knowing capabilities of transformers. Finally an extensive debugging process was performed, which uncovered a simple bug in the implementation of the transformer architecture. The problem lay in *pythonic* way of list object initialization. Instead of creating the required number of decoder and encoder blocks, the architecture contained a pointer to a single instance of decoder block, and a single encoder block. Fixing the mistake largely improved accuracy of the models.

Fixed number of 50 epoches for all models was chosen to, still operating under the assumption of transformer architecture needing more training steps than RNN. Results explained later in this work show, that after fixing the code bug deviating from the original training regime was not necessary to obtain similar scores to ones presented in the original paper.

2.4 BART Models

The models are the "standard" BART models using weights trained by the Facebook research team. In order to finetune the base model, the architecture is enhanced by a LM-head, as well three different adapters: LORA blocks, IA3, and bottleneck adapters. The training regime is identical to the one presented in the original paper. Data preprocessing is the standard processing technique for BART models obtained by using pretrained vocabulary and tokenizers. The models will be referred to as:

Bart: the baseline BART model using only a simple language head

BartLora: the baseline BART model using both a simple language head, and a LORA blocks

BartIA: the baseline BART model using both a simple language head, and a IA3 adapter

BartNeck: the baseline BART model using both a simple language head, and a bottleneck adapter

2.4.1 Language Head

The dimension of last hidden of a BART model is 768 for each input token. In order to map the output to BART's vocabulary (approx 50 thousands tokens) one would need to use at least one layer of dimension $768 * 50000$. In order to avoid the

large(approx. 20% of models size) language head a novel idea is introduced. Vocabulary is created containing all BART-tokens(byte-wise) needed to represent the output sequences of SCAN dataset. The size of the vocabulary is then used as the second dimension of the language head, which results in approx. dimensions of $768 * 50$. In training, and inference the models output translated between the two vocabularies as needed.

3 Results

OBS: Unfortunately, due to many unseen obstacles, BART models were not saved, and thus not evaluated, nor visualized. Due to time constraints I physically can not finish the individual part before the deadline. The attached notebook contain code for training the BART models, and simple inference. The `*metric-framework*` from the group part of the project could be now used to evaluate the models, and visualize the results automatically (see group project notebooks). I apologise, and I hope I can still achieve a passing grade.

The presented results were obtained after fixing the code bug, thus all models were trained for 50 epoches, and with teacher-forcing only. However the data collection happened every 5 epoches allowing us to analyse the results more in deep, and reflect about choices made.

The resulting accuracies are compared to a predefined baseline, and not to metrics from the original work.

3.1 Experiment 1

The obtained accuracies, as well as the baseline metrics can be observed in Figure 1. It is clear from the figure, that the vanilla models obtain lower accuracies than the baseline for less than 4% of training data. In this experiment 5 epoches equal approx. 10k steps(as in the original paper). One can observe, that the difference between the baseline and the obtained results is significantly larger for 5 epoch training. There is however small to none difference between 50 and 70 epoches.

3.2 Experiment 2

The obtained accuracies, as well as the baseline metrics can be observed in Figure 2 and Figure 3. In contract to Experiment E1 the accuracies for the model trained for 5 epoches(approx 10k steps) are on average higher than the baseline values, both for

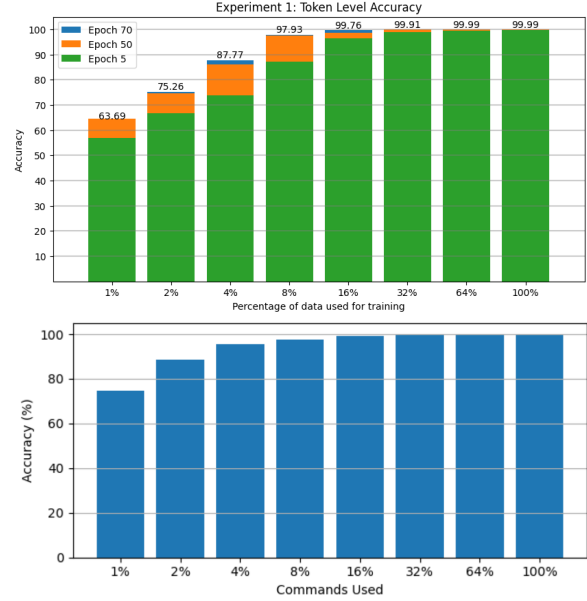


Figure 1: The upper plot shows the accuracies obtained by vanilla models. The lower plot represents the baseline metrics. In the upper plot, the values on top of bars are the accuracies for the highest epoch number.

the results using length oracle and without. In this case it is clear, the models trained for 50 and 70 epoches overfit the data, and did not generalized well for longer sequence lengths. This can be seen by looking at the accuracy scores on the bars, which indicate the accuracy for the 70 epoch model.

3.3 Experiment 3

The results for experiment 3 can be seen in figures 4, and 5. The results for experiment E3.1 i.e. few examples of "jump" primitive, can be observed in the two figures in from of dots. The baseline values were:

Turn Left: Sequence-level accuracy: 16.39%, Token-level accuracy: 77.98%, where the corresponding values obtained by vanilla models were approx 70% and 80% for 5 epoch model, and 100% and 100% for the 25 epoch models. The accuracies are higher, especially sequence-wise.

Token-level accuracies for experiment E3.2 show, that the vanilla models obtain higher accuracies for at least 16 instances of "complex context" of the primitive "jump", but they score lower for less than that. The only exception is the 0-shot case. One possible explanation would be, that the model overfits to the particular instances of "jump" in context when there are only few, whereas is generalizes much better without seeing it in any complex context.

Sequence-level accuracies as much higher than the baseline values, where the highest score is under 60% compared with vanilla model producing 97% accuracy. Especially cases with 8 and 16 complex examples are interesting. Here one can observe, that the baseline values are significantly lower than the ones obtained by vanilla models. This could point to the following conclusion: the vanilla models are better at predicting whole sequences correct, and on average when they fail they fail to predict most of the output sequence.

4 Discussion

The vanilla models trained in this work showed to be able to obtain similar, and sometimes significantly higher accuracies, than the baseline models. Even though the models were trained for many more training steps, than in the original paper, the analysis of results were performed for both 10 thousands steps, and the higher values. It is unclear how much of an impact had the decision of teacher-forcing only, but assuming the difference was not significant we can confirm our hypothesis. This is because the baseline results were better than the ones in the original paper, and they were created using Transformer architecture.

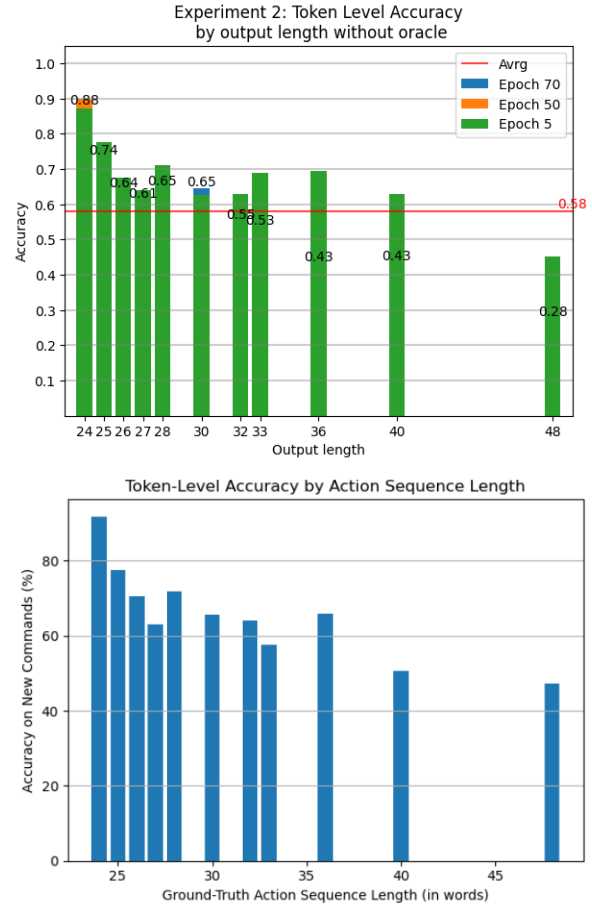


Figure 2: The upper plot shows the accuracies obtained by vanilla models. The lower plot represents the baseline metrics. In the upper plot, the values on top of bars are the accuracies for the highest epoch number.

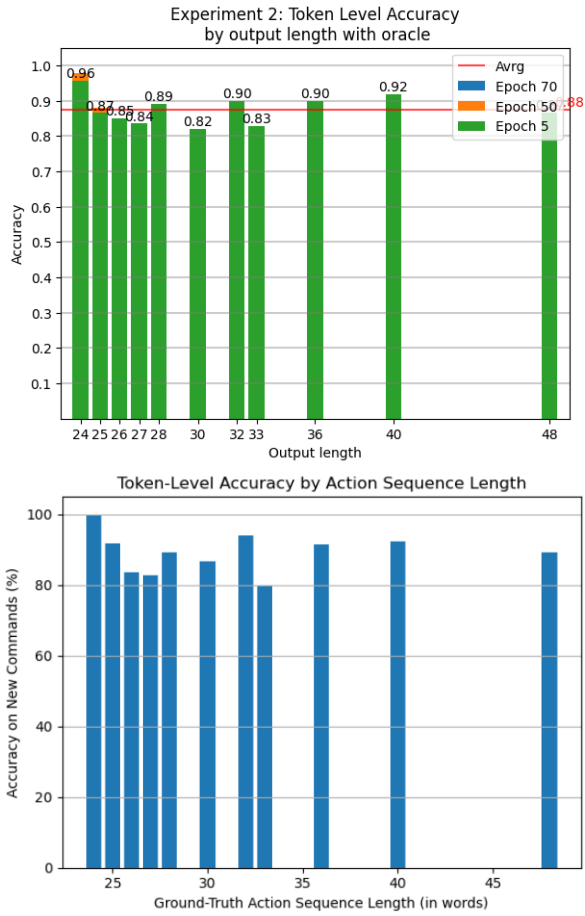


Figure 3: The upper plot shows the accuracies obtained by vanilla models. The lower plot represents the baseline metrics. In the upper plot, the values on top of bars are the accuracies for the highest epoch number.

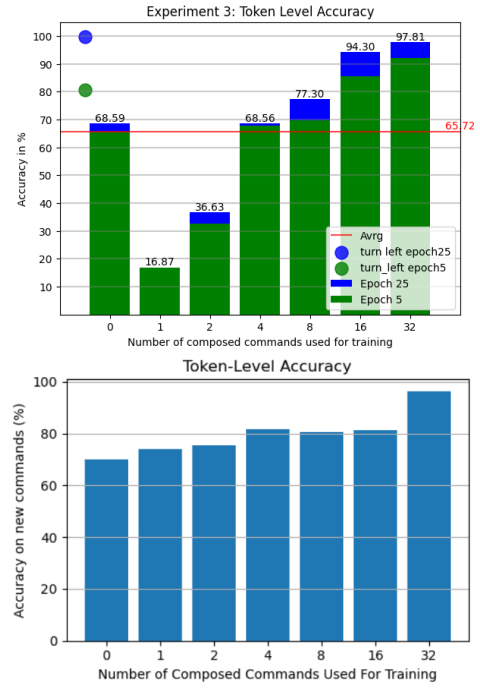


Figure 4: The upper plot shows the accuracies obtained by vanilla models. The lower plot represents the baseline metrics. In the upper plot, the values on top of bars are the accuracies for the highest epoch number.

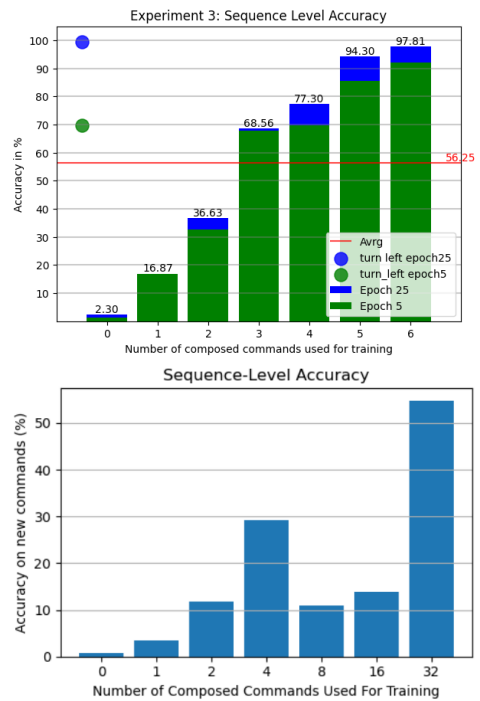


Figure 5: The upper plot shows the accuracies obtained by vanilla models. The lower plot represents the baseline metrics. In the upper plot, the values on top of bars are the accuracies for the highest epoch number.